

Project Proposal: TraceAI - The AI Answer Verification and Evidence Auditor

Title: TraceAI: A Human-in-the-Loop Full-Stack Platform for Claim-Level Verification and Evidence-Based Reliability Analysis of Generative AI Responses

Introduction and Motivation

The rapid adoption of Generative AI (Gen-AI) systems such as ChatGPT, Gemini, Claude, and other large language models has made AI-generated answers widely available in education, software development, research, and everyday decision-making. However, these systems can produce fluent statements that are inaccurate, unsupported, contradictory, or outdated. Because incorrect claims are often written confidently, users may find it difficult to determine which parts of an answer can be trusted.

Existing fact-checking tools are often designed for human-written news or depend on proprietary services that provide limited visibility into how evidence was selected. This project proposes TraceAI, a human-in-the-loop full-stack platform that accepts an AI-generated response and supporting documents, extracts factual claims, retrieves relevant evidence, and classifies each claim as supported, partially supported, contradicted, or unverifiable. Human reviewers can inspect the evidence and correct the automated result, producing transparent analyses and evaluation data.

Research Questions

1. **System Design:** How can a robust, intuitive, and secure full-stack architecture be designed to support AI-response submission, evidence-document upload, claim-level analysis, and transparent result presentation?
2. **Claim Extraction:** What natural-language-processing approach can effectively separate an AI-generated response into factual claims while excluding opinions, instructions, and non-verifiable statements?
3. **Evidence Retrieval and Verification:** How can semantic retrieval and language-inference methods identify relevant evidence and classify claims as supported, partially supported, contradicted, or unverifiable?
4. **Human Feedback and Transparency:** How can reviewer corrections, confidence scores, and evidence passages be presented to create an understandable human-in-the-loop verification process?

Project Objectives

The primary objective is the design, implementation, and evaluation of the TraceAI full-stack application.

Component	Objective	Technologies (Example)
Full-Stack Development	Develop a functional, scalable web application covering the frontend, API, AI service, and database layers.	Frontend: React and TypeScript; API: Node.js, Express or NestJS; AI Service: Python and FastAPI.
Document & Evidence Management	Implement secure upload, text extraction, chunking, storage, and retrieval for PDF and text evidence documents.	Multer; PDF text extraction; object storage or local filesystem; PostgreSQL for metadata.
AI Claim Analysis Service	Extract factual claims, generate embeddings, retrieve relevant passages, and classify the relationship between each claim and its evidence.	spaCy or Transformers; Sentence Transformers; Hugging Face models; FastAPI; pgvector.
Human Review & Feedback	Allow users to inspect evidence, confirm or correct automated classifications, and record structured reviewer feedback.	JWT authentication; role-based authorization; validated review schema; audit history.
Visualization Dashboard	Build an interactive dashboard showing reliability scores, verification outcomes, model comparisons, and human-AI agreement.	Recharts or Chart.js; REST API; Python Pandas for aggregated analysis.

Methodology (Full-Stack and AI Focus)

Frontend Development

- **User Interface:** A responsive and accessible interface for submitting an AI-generated answer, uploading evidence, and reviewing claim-level results.
- **Analysis Workflow:** A guided, multi-step process to ensure that all information required for verification is captured:
 1. **Response Submission:** Enter the original prompt, AI-generated answer, model name, and topic.
 2. **Evidence Upload:** Upload one or more PDF or text documents that will act as the verification source.
 3. **Automated Analysis:** Display extracted claims, retrieved passages, classification labels, and confidence scores.

- 4. Human Review: Confirm, correct, or request more evidence for each automated result.

AI and Backend Development

- API Design: Develop secure RESTful APIs for user management, document ingestion, analysis creation, result retrieval, and review submission.
- AI Pipeline: Implement claim extraction, document chunking, embeddings, semantic retrieval, classification, and transparent reliability scoring.
- Security: Apply input validation, authorization, rate limiting, safe file handling, and privacy controls for uploaded documents and analysis records.

Database Design

- Schema: Design a relational database schema to efficiently store:
 - Users and roles (hashed passwords, permissions, timestamps).
 - Analyses and AI responses (prompt, response, model, topic, status).
 - Documents, passages, and embeddings (file metadata, text chunks, vector references).
 - Claims, verification results, evidence links, confidence scores, and human reviews.

Evaluation

The project will be evaluated on two fronts:

1. Technical Evaluation: System stability, API latency, document-processing time, retrieval speed, security controls, automated tests, and code quality.
2. AI and Usability Evaluation:
 - Model Evaluation: Measure claim-extraction precision and recall, evidence-retrieval relevance, classification accuracy/F1, and human-AI agreement on a labelled test set.
 - User Study: Conduct a small usability study to assess the clarity of the workflow, evidence presentation, reliability score, and review controls.

Expected Contribution

This project will contribute to software development and applied AI by:

- Software Artifact: Delivering a functional, open-source TraceAI application that combines a modern full-stack architecture with an independent Python AI service.
- Explainable Verification Pipeline: Providing claim-level labels together with the retrieved evidence and confidence information used to produce each result.
- Evaluation Dataset: Generating a structured dataset of AI claims, evidence passages, automated classifications, and human corrections for future experimentation.

Timeline (Suggested Milestones)

Phase	Milestone	Duration
Phase 1: Planning & Setup	Literature Review, Requirements Gathering, AI Pipeline Design, Core Architecture Design (Tech	4 Weeks

	Stack Selection), Database Schema, and Evaluation Plan.	
Phase 2: Core Development	User Authentication, Analysis API, Evidence Upload, Basic Frontend UI, Document Extraction, and Initial Database Implementation.	8 Weeks
Phase 3: AI Verification Pipeline	Claim Extraction, Embedding Generation, Vector Retrieval, Claim-Evidence Classification, and Reliability Scoring.	6 Weeks
Phase 4: Review, Dashboard & Evaluation	Human Review Workflow, Interactive Dashboard, System Testing, Model Evaluation, and Usability Study.	6 Weeks
Phase 5: Documentation & Deployment	Code Documentation, Docker Configuration, Deployment, Final Report, README, and Demonstration Preparation.	8 Weeks